



PhD Thesis Artifact:

StarMalloc: Formal Verification and Evaluation of a Modern, Hardened Memory Allocator

Antonin Reitz, Inria

A Artifact Appendix

A.1 Abstract

The artifact contains the formally verified implementation of the StarMalloc modern, hardened C userspace memory allocator. This includes the proof development, corresponding extracted C code and the resulting memory allocator, as well as benchmarks used for evaluation: the mimalloc-bench benchmarking suite and a specific build of Firefox.

A.2 Description & Requirements

A x86_64 computer equipped with the Linux operating system is required to run this artifact. In addition to that, Nix and a GUI (such that Firefox may be launched) must be installed.

A.2.1 Security, privacy, and ethical concerns

None.

A.2.2 How to access

The artifact can be accessed at the following addresses:
<https://github.com/Inria-Prosecco/StarMalloc/tree/artifact> or <https://doi.org/10.5281/zenodo.18357583>.

A.2.3 Hardware dependencies

A computer with 16GB of RAM and 50GB of free disk space is required to evaluate this artifact. Linux required configuration includes temporarily setting the `vm.max_map_count` value to 1048576 by running the following command:
`# sysctl vm.max_map_count=1048576`.
Configuring `zram` and mounting `/tmp` as a `tmpfs` may be useful.

A.2.4 Software dependencies

All software dependencies are set-up using Nix. Thus, only Nix and a GUI (such that Firefox may be launched) are required to evaluate this artifact.

A.2.5 Benchmarks

Benchmarks include the `mimalloc-bench` benchmarking suite to test performance, as well as a specific build of the Firefox browser to test applicability to real-world workloads.

A.3 Set-up

A.3.1 Installation

[5 human-minutes + 1 compute-hour + ~20GB disk]

Using Nix, from the root of the downloaded artifact, the following commands can be run to compile all dependencies and enter a shell with a correct environment.

```
cd StarMalloc
nix develop
```

In case of a failure related to CMake: exit the shell, `rm -rf extern` and reenter the shell through `nix develop`.

A.3.2 Basic Test

Inside a correct environment, to compile pre-extracted files:

```
make light
```

A.4 Evaluation workflow

A.4.1 Major Claims

- (C1): StarMalloc has been formally verified using the Steel framework, and thus comes with functional correctness theorems for provided APIs (e.g., `malloc`), as well as configurable security mechanisms (e.g., guard pages). This is proven by experiment (E1).
- (C2): StarMalloc supports all benchmarks included in the `mimalloc-bench` benchmarking suite and exhibits competitive performance with respect to `hardened_malloc`. This is proven by experiment (E2).
- (C3): StarMalloc can be used as a drop-in replacement for the Mozilla Firefox browser, e.g., by testing (a specific build of) Firefox when using StarMalloc as the allocator on the JetStream or Speedometer browser benchmarks. This is proven by experiment (E3).

A.4.2 Experiments

(E1): [30 human-minutes + 30 compute-minutes + 1GB disk]

Check that correctness theorems about provided APIs stated in the paper and the thesis are included in the artifact, and that those can be mechanically checked by the Steel verification toolchain.

Preparation: Set up a correct environment using Nix.

Execution: Go in the `StarMalloc` repository and run the `make -j 3` command: this will verify all of `StarMalloc` using the Steel verification toolchain. Theorems about provided APIs stated in the paper and specifications about configurable security mechanisms are listed in the `README` file (section “Artifact: experiment (E1)”), with precise references to corresponding code. Check that they correspond to the prose.

Results: Verification should be successful, extracted files should match those tracked by git in the `dist/` folder, compilation to the `out/starmalloc.so` should be successful, theorems should match the content of the paper and the thesis.

(E2): [10 human-minutes + 1 compute-hour + 1GB disk]

Check `StarMalloc`’s performance on `mimalloc-bench` benchmarks, using `hardened_malloc` as a baseline.

Preparation: Set up a correct environment using Nix.

Execution: Run the `make N=5 bench` command. This will benchmark the system allocator (`sys`), `hardened_malloc` (`hm`) as well as `StarMalloc` (`st`) on all `mimalloc-bench` benchmarks.

`N=5` means that each execution of a benchmark using a given memory allocator will be repeated 5 times: each repetition of all benchmarks on all memory allocators require roughly 12 compute-minutes on a recent 16-core machine.

Results: Results are placed inside the `bench/` folder, inside a subfolder whose name is the date when the experiment was launched. This includes the CSV file with all results as well as PDF files including results in the form of tables. `hardened_malloc` is used as a baseline, results are given as the geometric mean of all runs.

Results should be consistent from run to run and in the 0.70x-1.30x range. The geometric mean of the overhead of using `StarMalloc` wrt to using `hardened_malloc` on all benchmarks should be roughly equal to 1.

(E3): [20 human-minutes + 1 compute-hour + ~30GB disk]

Check that `StarMalloc` can be used as a drop-in replacement memory allocator for the Firefox browser, using a specific build of Firefox.

Preparation: Set up a correct environment using Nix. Run the `make firefox-nojemalloc` command. This will build a specific version of Firefox that is required for the experiment. Close any opened Firefox process.

Execution: Run the following command:

```
LD_PRELOAD=./out/starmalloc.so
./result/bin/firefox.
```

A Firefox window should open. Manually open the <https://browserbench.org/> URL. Test any of the proposed benchmarks, e.g., the `JetStream2.2` benchmark.

Results: Firefox should be working as expected.

Addendum: To confirm that `StarMalloc` is indeed used during the entire execution of benchmarks (and not only at Firefox startup), the following can be used.

Build a specific version of `StarMalloc` with deallocations logs: `make light-logs`. Close any opened Firefox process. Run the following:

```
LD_PRELOAD=./out/starmalloc-logs.so
./result/bin/firefox.
```

Repeat the rest of the experiment (due to logs printed in the terminal, this may be slightly slower, even though 1% of all deallocations are logged). As long as Firefox is running using `StarMalloc`, logs should continuously appear; closing Firefox will stop such logs to be appended at the end of the file.

Finally, we note that a specific build of Firefox is required since Firefox uses by default the `mozjemalloc` memory allocator in a non-overridable way.

A.5 Notes on Reusability

`StarMalloc` can be used in various environments regarding security and performance constraints, as it is configurable. To do so, one can edit configuration values in `src/Config.fst`, e.g., by modifying the `guard_pages_interval` value from `2sz` to `8sz`. Running `make` will reverify incrementally only a few files, extract corresponding updated C code to the `dist/` folder and compile it to the `out/starmalloc.so` file. Once this file is copied to the `extern/mimalloc-bench/extern/st` directory, this different version of `StarMalloc` can be benched using `mimalloc-bench`.

Leveraging the project architecture and iterative development enabled by our verification methodology, one could implement additional security mechanisms (e.g., for large allocations) or refine existing ones (e.g., the quarantine, see PR).

This is all the more true since a lot of efforts have been put into proof maintenance over the past few years, so that `StarMalloc` supports the latest versions of `F*`, `Steel` and `KaRaMeL` as of 2026-01-23. It should be possible to pursue preliminary work for 64K system pages and enable support of `StarMalloc` for other architectures (e.g., ARM), as well as for Android (e.g., by reusing `hardened_malloc` stubs for Android).

Finally, we expect our work around the `nix-mimalloc-bench` repository¹ that enables one to use the `mimalloc-bench` benchmarking suite through Nix

¹ Accessible at <https://github.com/cmouvcc/nix-mimalloc-bench>.

(which was very useful for this artifact) to be reusable by others.

A.6 Some help with this artifact and Nix

Information about all directories contained in this artifact is included in the README.md file. Nix is required so that experiments requiring a GUI may be used easily (this is non-trivial using Docker).

Once Nix is installed, it is required to enable the `nix-command` and `flakes` experimental features, e.g., through edits to the `/etc/nix/nix.conf` file. Rootless Nix may not be allowed: in that case, `nix develop` commands should be prefixed by the `sudo` command.

Entering a correct shell can be done using the `nix develop`. Beyond setting a myriad of shell values and building or fetching any required software dependency, this command build (if not already built) `mimalloc-bench` and copies the result in the `extern/` directory, that should be removed between executions of the `nix develop` command.

Nix helps with reproducibility, but is not always easy to use. `nix develop` should be the only Nix command that you will need to use to evaluate the artifact. Once you are done evaluating the artifact, you can remove the artifact directory and the Nix store used as a cache can then be cleared using `$ nix-collect-garbage -d`. Proceed with caution, this affect the Nix store *globally* for the considered user.

A.7 Acknowledgments

This work has been enabled by a fruitful collaboration with Aymeric Fromherz and Jonathan Protzenko.

A.8 Version

Based on the LaTeX template for Artifact Evaluation V20231005. Submission, reviewing and badging methodology followed for the evaluation of this artifact can be found at <https://secartifacts.github.io/usenixsec2026/>.